

Scaling Recommender Systems: Retrieval and Ranking 🚀

When you have a catalog of **millions** of items (songs, videos, products), it is computationally impossible to run a complex neural network prediction for *every* single item every time a user visits your site.

To solve this, large-scale systems split the process into two steps: **Retrieval** and **Ranking**.

Step 1: Retrieval (Candidate Generation)

The goal of this step is fast, broad coverage. We want to quickly narrow down millions of items to a manageable list of candidates (e.g., hundreds) that contain at least *some* items the user will like. It is okay if this list includes some bad recommendations, as they will be filtered out later.

Common Retrieval Strategies:

- **Similar to Recent History:** For the last 10 items the user engaged with, look up the "most similar items" (using pre-computed movie vectors v_m).
- **Top Genres:** If the user loves "Sci-Fi," retrieve the top 10 most popular Sci-Fi movies.
- **Demographics:** Retrieve the top 20 most popular movies in the user's country.

Result: A list of perhaps 500–1000 plausible candidates. (Duplicates and items already watched are removed).

Step 2: Ranking

The goal of this step is accuracy. We take the small list of candidates from the retrieval step and use our powerful Neural Network model to rank them.

The Process:

1. **Compute User Vector:** Run the user's features through the User Network to get v_u . (Done once).
2. **Predict Ratings:** For each of the ~500 candidates, perform the inference (dot product $v_u \cdot v_m$) to predict the user's specific rating or probability of clicking.
3. **Sort:** Order the items by their predicted score.
4. **Display:** Show the top 10–20 items to the user.

[Image of anomaly detection plot]

Performance Tuning

A key decision in this architecture is **how many items to retrieve** in Step 1.

- **Retrieving More Items (e.g., 1000):**

- *Pros:* Better chance of finding the absolute best content (higher accuracy).
- *Cons:* Slower system performance (higher latency).

- **Retrieving Fewer Items (e.g., 100):**

- *Pros:* Very fast.
- *Cons:* Risk of missing the best content because it was never retrieved for ranking.

Recommendation: Perform offline experiments. Check if increasing the retrieval count significantly improves the quality of the final top-10 list. If the improvement is small, stick to a smaller number for speed.

Summary of Recommender Systems Architecture

1. **Catalog:** Millions of items.
2. **Retrieval:** Fast heuristics filter this down to ~500 candidates.
3. **Ranking:** Neural Networks predict precise scores for those ~500.
4. **Output:** Top 10 items shown to the user.